

AlphaZero Chess Raport

Project Overview

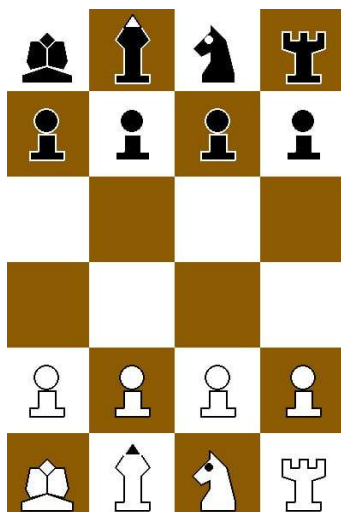
In this report, we present the implementation of a self-learning AlphaZero-based algorithm designed to play chess on a custom board. Our project combines Monte Carlo Tree Search (MCTS) with a neural network trained entirely through self-play, allowing the system to learn effective strategies without any predefined knowledge of openings or tactics.

Algorithm Development steps:

1. Understanding of provided Chess interface
2. Determining potential useful libraries
3. Creating MCTS algorithm
4. Creating Alpha Zero Loop
5. Creating Neural Network
6. Performing Neural Network Learning
7. Evaluating the results
8. Neural Network adjustments and repentance of steps 6-8
9. Final Version of the algorithm

1. Understanding of provided Chess interface

We are using the functions and the rules given by lecturer with no significant changes. The important changes are that due to small board the ability to castle is disabled.



2. Determining potential useful libraries

In the project we will be using:

- NumPy – for efficient numerical computations
- PyTorch – building and training the neural network.
- Matplotlib – for visualization (e.g., training curves)
- Pygame – used in the interface for drawing the board

We've also considered chess-related libraries, but since we decided to use interface provided by Lecture we don't need them anymore.

3. Creating MCTS algorithm

File: *mcts.py*

We use the MCTS class to simulate future moves and pick the best one. Each position is a *MCTSNode* that stores the state, player, visits, value, and possible next moves. In each simulation We:

1. Select: We move down the tree by picking the child with the highest score using the PUCT formula (in our code variable *ucb*).
2. Evaluate: For a new position, we encode the board with *encode_state()* and pass it to the neural network to get move probabilities and a value.
3. Expand: We create new child nodes for all legal moves and assign priors from the policy output.
4. Backpropagate: We update visit counts and values going back to the root.

After all simulations, we pick the move from the root with the most visits.

4. Creating Alpha Zero Loop

File: *alpha_zero_loop.py*

We run 10 training cycles. In each cycle we:

1. Play 30 self-play games using *Strategy_MCTS* and our custom 6×4 board.
2. Save the game data (states, policies, values).
3. Train the ChessNet model for 5 epochs using this data.
4. Save the updated model after each cycle.

5. Creating Neural Network

File: *ChessGame.py*

Input:

The input to our neural network consists of 12 channels, each representing the presence of a specific type of chess piece on the board. The first six channels correspond to white pieces: pawns, knights, bishops, rooks, queens, and kings. The next six channels represent the same piece types for black. Each channel is a 2D matrix with the same dimensions as the board, where a value of 1.0 indicates the presence of that piece on a given square, and 0.0 means the square is

empty for that type. This results in a one-hot encoded 3D tensor of shape (rows, cols, 12), which is then permuted into (12, rows, cols) when passed into the neural network. This encoding allows the model to process the full game state as a stack of binary maps, each corresponding to a particular piece type and color.

Network and output:

We use PyTorch to define and train our neural network model. The network is implemented as a subclass of `nn.Module` and processes the input board state using convolutional layers. It outputs two separate predictions: a policy vector representing probabilities for all possible moves, and a value estimating the expected outcome of the game. Training is done using supervised learning from self-play data, with cross-entropy loss for the policy output and mean squared error for the value output. The optimization is performed using the Adam optimizer.

6. Performing Neural Network Learning

File: *alpha_zero_loop.py*

The model is trained using data generated from self-play games, where each training example consists of a board state, a target move distribution (policy), and an expected game result (value). During training, these are fed in batches into the neural network. The network outputs a predicted policy (move probabilities) and a predicted value (game outcome). Two separate loss functions are used: cross-entropy loss between the predicted and target policies, and mean squared error between the predicted and actual values. The total loss is the sum of both. The model parameters are updated using the Adam optimizer. This process is repeated for several epochs in each cycle, allowing the model to improve based on its own gameplay data.

7. Improvements

In order to increase performance of our model we introduced improvements:

High diversity of moves in the beginning and cooling down

```
def apply_temperature(probs, temperature):
    if temperature <= 0:
        one_hot = np.zeros_like(probs)
        one_hot[np.argmax(probs)] = 1
        return one_hot
    adjusted = np.power(probs, 1.0 / temperature)
    adjusted /= np.sum(adjusted)
    return adjusted

def dynamic_temperature(step, max_steps):
    if step < max_steps * 0.3:
        return 3.5
    elif step < max_steps * 0.7:
        return 2
    else:
        return 1
```

Randomized opening

That will ensure that the model is presented with variations of the states it can learn from. It was such an important thing for us (due to the small board) that we also introduced another randomization regarding the first few moves of each play. There are between 0 and 20 random moves at the beginning of each game.

```
RANDOM_OPENING_MOVES = (0, 20)
def play_random_opening_moves(game, state, player, moves):
    for _ in range(moves):
        legal = game.actions(state, player)
        if not legal:
            break
        action = random.choice(legal)
        state, _ = game.next_state_and_reward(player, state, action)
        player = 3 - player
    return state, player
```

Material difference reward

We've started rewarding the model for creating a positive difference in material value to encourage it to make captures. That was a really big change in terms of the model's behavior, which can be observed mostly with high-value pieces. Unfortunately, it is not as effective when it comes to capturing pawns.

```
def material_balance(state):
    white_total, black_total = 0, 0
    board_array = state.Board
    for row in board_array:
        for cell in row:
            if isinstance(cell, str):
                if cell.isupper():
                    white_total += material_value(cell)
                elif cell.islower():
                    black_total += material_value(cell)
    return white_total - black_total
```

Explicit capture possible information

Additionally, we decided to provide the model with explicit information about possible captures in order to further encourage it to make them. We are aware that this is rather a short-term strategy, prone to traps and similar risks, but we believe it is still valuable to introduce such information.

```
def is_capture_action(state, action):
    target_row, target_col = action[2], action[3]
    board = state.Board
    if 0 <= target_row < len(board) and 0 <= target_col < len(board[0]):
        target_cell = board[target_row][target_col]
        return isinstance(target_cell, str) and target_cell != '.'
    return False
```

8. Final Model

The final model was trained on $18 \times 100 + 12 \times 80$ games, and we are satisfied with the final results, although we are aware that it is not yet perfect. The model performs very well when it comes to capturing exposed pieces, especially those with higher value like rooks or queens. However, it struggles more with capturing pawns. Sometimes, it makes seemingly meaningless moves, such as moving a rook one square to the side and then back again. That said, in positions without an obvious capture, it might not be a bad strategy to preserve the possibility of advancing pawns at any time—since they cannot move backward. Some of the model's limitations may be due to hardware constraints (both of us are using laptops without dedicated GPUs), which significantly impact training time.